



Le Framework PHP

Laravel

Pr. Abdelali El Gourari

Plan: Laravel

- *Introduction*
- *MVC*
- *Composer*
- *Installation de Composer*
- *Installation de Laravel*
- *Structure des dossiers*
- *Relier la base de données*
- *Créer les models*
- *Controllers*
- *Routes*
- *Créer la vue*

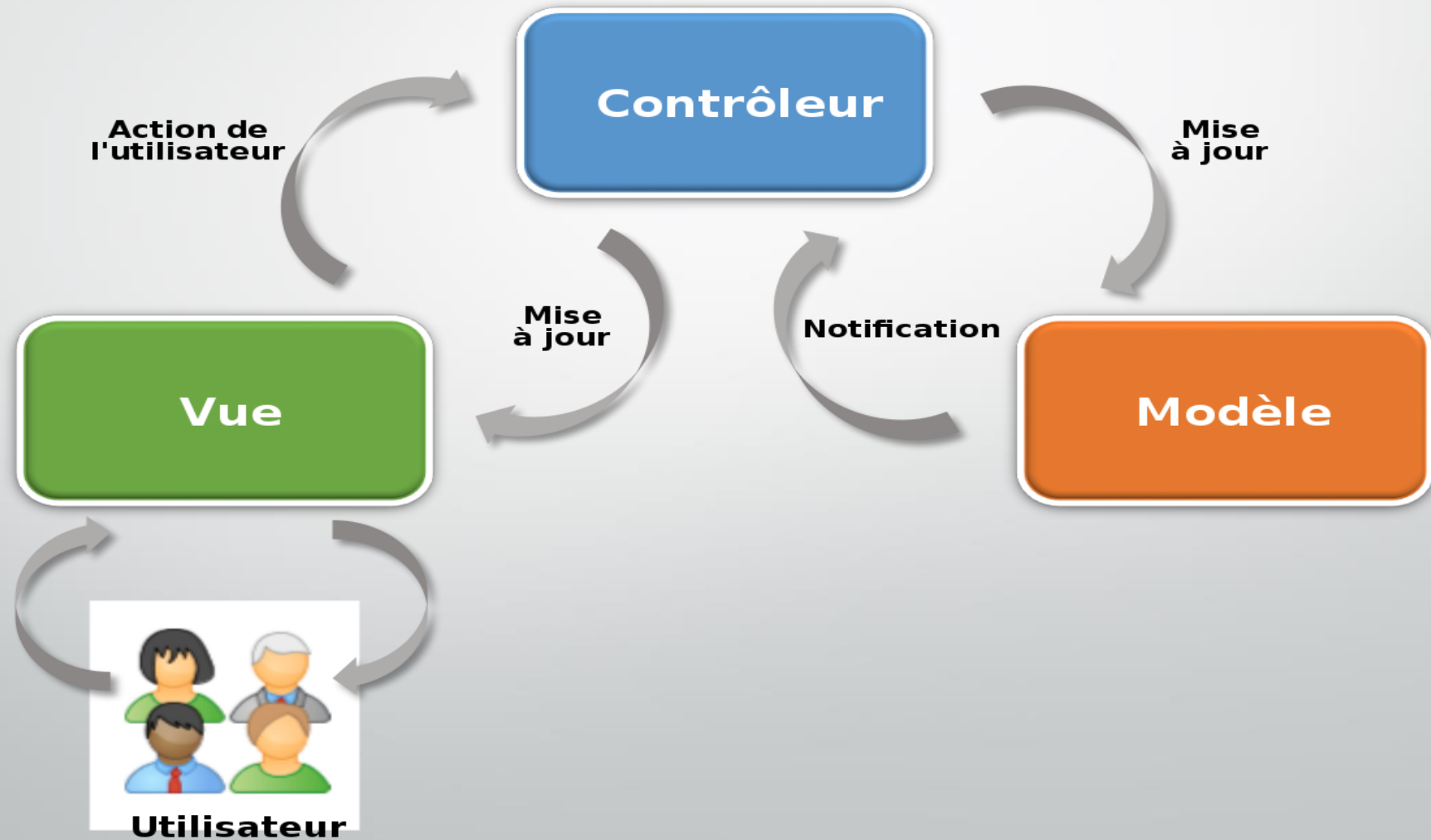


Introduction

- *Laravel est un Framework PHP libre de droits qui a fait son apparition en 2011.*
- *L'un des meilleurs frameworks avec Symfony et CodeIgniter.*
- *Laravel utilise la toute dernière version de PHP et a fréquemment des patches de disponibles avec de nouveaux éléments et des mises à jour qui règlent les problèmes, ce qui prouve qu'il est en constante évolution et amélioration.*
- *il se base sur « Composer », le meilleur outil de dépendance qui gère des projets en PHP jusqu'à maintenant.*

MVC

Laravel se base effectivement sur le patron de conception MVC, c'est-à-dire modèle, vue et contrôleur.



Composer

- La première étape serait d'installer le Composer dont Laravel utilise.
- Qu'est ce qu'un composer? Composer trouve les fichiers PHP qu'on a besoin dans un projet.
- Il va les chercher et les installer à la bonne place.
- Comme Laravel est un Framework PHP, il est très pratique de l'utiliser pour bien partir un projet.



Installation de Composer

- On peut le télécharger sur le site getcomposer.org ou directement sur le site de laravel.com.
- Ce Composer a une entente avec Laravel et va chercher tous les fichiers que Laravel utilise pour bien fonctionner.
- En installant le Composer, on pourra installer beaucoup plus facilement et rapidement le Framework Laravel, sans faire d'erreur.
- Une fois le fichier « Composer-Setup.exe » est installé dans l'ordinateur, on peut maintenant installer Laravel.



Installation de Laravel

- Il faut savoir où installer le projet. Je vous conseille de travailler en localhost, avant de travailler sur le serveur lui-même, pour simple efficacité.
- Ouvrez une fenêtre Windows de où vous voulez mettre le dossier de projet et faites click-droit, si vous avez bien installé le Composer, vous devriez voir dans la liste « use Composer here ».
- Après avoir installé PHP et Composer, vous pouvez créer un nouveau projet Laravel via la commande Composer create-project :

`composer create-project laravel/laravel your-project-name`

Installation de Laravel

- Composer : signifie qu'on utilise le Composer.
- create-project : signifie qu'on crée un projet.
- laravel/laravel : va chercher les fichiers à installer pour le Framework Laravel.
- your-project-name : on met le nom qu'on veut donner à notre projet.

Vous pouvez également créer de nouveaux projets Laravel en installant globalement le programme d'installation de Laravel via Composer :

- ✓ **composer global require laravel/installer**
- ✓ **laravel new your-project-name**

Installation de Laravel

- Après la création du projet, démarrez le serveur de développement local de Laravel à l'aide de la commande serve de la CLI Artisan de Laravel :
 - ✓ `cd your-project-name`
 - ✓ `php artisan serve`

Une fois que vous avez démarré le serveur de développement Artisan, votre application sera accessible dans votre navigateur Web à l'adresse `http://localhost:8000`.

Artisan !

- **Artisan** est une interface de ligne de commande pour **Laravel**
- Commandes qui font économiser **le temps**
- Générer des fichiers avec Artisan est **recommandé**
- L'exécution de `php artisan list` dans la console

app	Set the application namespace
auth	
auth:clear-resets	Flush expired password reset tokens
cache	
cache:clear	Flush the application cache
cache:forget	Remove an item from the cache
cache:table	Create a migration for the cache database table
config	
config:cache	Create a cache file for faster configuration loading
config:clear	Remove the configuration cache file
db	
db:seed	Seed the database with records
event	
event:generate	Generate the missing events and listeners based on registration
ide-helper	
ide-helper:generate	Generate a new IDE Helper file.
ide-helper:meta	Generate metadata for PhpStorm
ide-helper:models	Generate autocompletion for models
key	
key:generate	Set the application key
make	
make:auth	Scaffold basic login and registration views and routes
make:command	Create a new Artisan command
make:controller	Create a new controller class
make:event	Create a new event class
make:job	Create a new job class
make:listener	Create a new event listener class
make:mail	Create a new email class
make:middleware	Create a new middleware class
make:migration	Create a new migration file
make:model	Create a new Eloquent model class
make:notification	Create a new notification class
make:policy	Create a new policy class
make:provider	Create a new service provider class
make:request	Create a new form request class
make:seeder	Create a new seeder class
make:test	Create a new test class
migrate	
migrate:install	Create the migration repository
migrate:refresh	Reset and re-run all migrations
migrate:reset	Rollback all database migrations
migrate:rollback	Rollback the last database migration
migrate:status	Show the status of each migration
notifications	
notifications:table	Create a migration for the notifications table
queue	
queue:failed	List all of the failed queue jobs
queue:failed-table	Create a migration for the failed queue jobs database table
queue:flush	Flush all of the failed queue jobs

Structure des dossiers

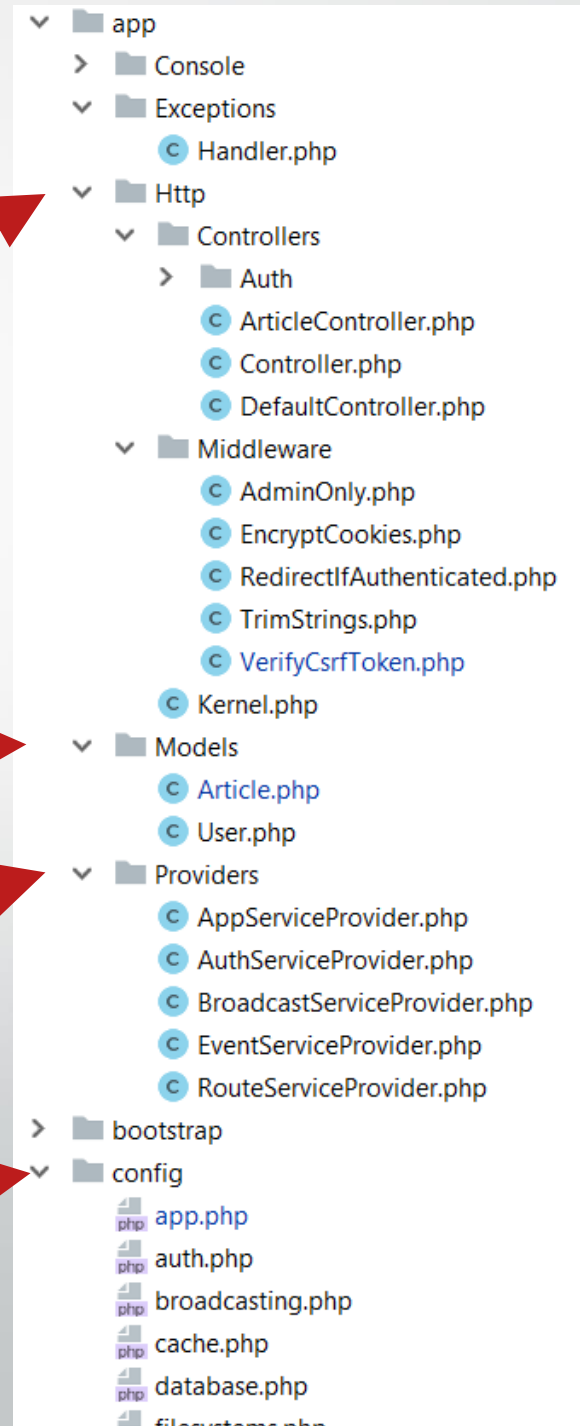
Il est important de d'abord analyser la structure des dossiers pour savoir comment la hiérarchie fonctionne.

Le dossier **app/Http** contient le fichier **Controllers**, **Middlewares** et **Kernel**.

Tous les modèles doivent être situés dans le dossier **app/Models**.

Les fournisseurs de services qui amorcent les fonctions de notre application sont situés dans le dossier **app/Providers**.

Tous les fichiers de configuration sont situés dans le dossier **app/config**.



Le dossier **Database** contient les **migrations** et **seeds**

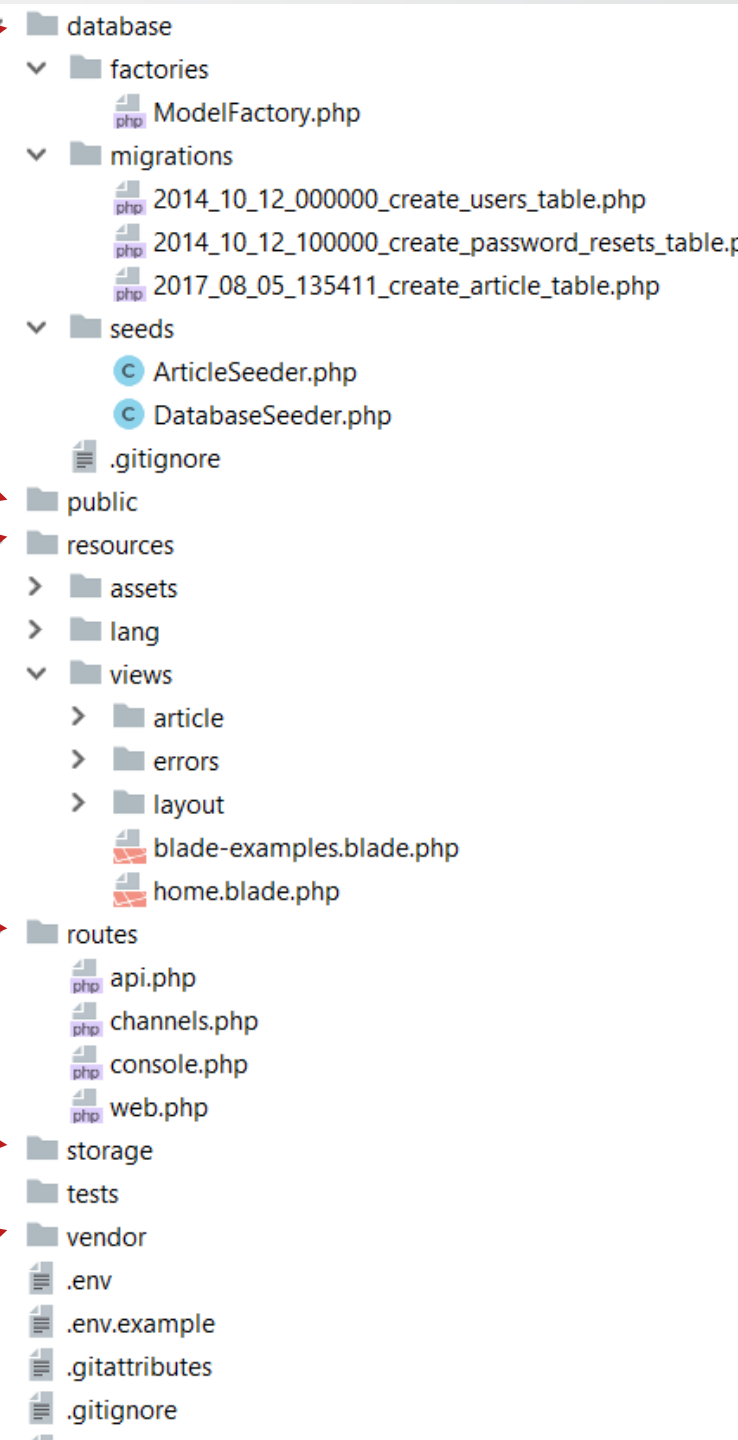
Le dossier **public** est le dossier réel que vous ouvrez sur le serveur Web.
Tous les fichiers JS / CSS / Images / Uploads sont placés là.

Le dossier des ressources contient toutes les **traductions**, les **vues** et les **actifs** (SASS, LESS, JS) qui sont compilés dans le dossier public.

Le dossier **routes** contient toutes les routes du projet.

Tous les **journaux / fichiers** de cache sont situés dans le dossier de **stockage**

Le dossier **vendor** contient tous les paquets (dépendances) du compositeur.



Relier la base de données

il faut relier la base de données au projet. Dans le dossier **config**, il existe le fichier **database.php**. En l'ouvrant, on peut voir plusieurs array, dont un qui est associé à mysql.

```
'mysql' => array(  
    'driver'      => 'mysql',  
    'host'        => 'localhost',  
    'database'    => '',  
    'username'    => '',  
    'password'    => '',  
    'charset'     => 'utf8',  
    'collation'   => 'utf8_unicode_ci',  
    'prefix'      => '',  
)
```

Relier la base de données

Pour 'database' => il faut mettre ensuite le nom de la base de données, soit 'portfolio' dans mon cas.

Pour 'username' => notre nom d'utilisateur de la connexion à phpmyadmin, qui est 'root' pour moi.

Pour 'password' => le mot de passe.

Et pour le charset, il est par défaut en utf8.

Comme il y a plusieurs type de connexion à une base de données disponible, il ne faut pas oublier de mettre par défaut le type de connexion que l'on va utiliser, soit 'mysql' comme ci-dessous, toujours dans le même fichier :

Routage

Le routage dans Laravel vous permet d'acheminer toutes les demandes de votre application vers leur contrôleur approprié. Les routes principales et primaires de Laravel reconnaissent et acceptent un URI (Uniform Resource Identifier) accompagné d'une fermeture, étant donné qu'il doit s'agir d'un moyen simple et expressif de routage.

Routing

Le fichier route de l'application est défini dans le fichier **routes/web.php**. Le routage général dans Laravel pour chacune des requêtes possibles ressemble à ceci :**http://localhost/**

```
Route::get('/', function () {  
    return 'Welcome to index';  
});
```


Routing

<http://localhost/user/dashboard>

```
Route::post('user/dashboard', function () {  
    return 'Welcome to dashboard';  
});
```

<http://localhost/user/add>

```
Route::put('user/add', function () {  
    //  
});
```

<http://localhost/post/example>

```
Route::delete('post/example', function () {  
    //  
});
```

Le mécanisme de routage dans Laravel

Le mécanisme de routage se déroule en trois étapes différentes :

- Tout d'abord, vous devez créer et exécuter l'URL racine de votre projet.
- L'URL que vous exécutez doit correspondre exactement à la méthode définie dans le fichier **root.php**, et elle exécutera toutes les fonctions connexes.
- La fonction invoque les fichiers de modèle. Elle appelle ensuite la fonction `view()` avec le nom du fichier situé dans **resources/views/**, et élimine l'extension de fichier **blade.php** au moment de l'appel.

Le mécanisme de routage dans Laravel

Routes/web.php

```
<?php Route::get('/', function () { return view('abdelali'); });
```

resources/view/abdelali.blade.php

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>ESMA_GI4</title>
```

```
  </head>
```

```
  <body>
```

```
    <h2>My name is Abdelali</h2>
```

```
    <p>Welcome to Laravel</p>
```

```
  </body>
```

```
</html>
```

Le mécanisme de routage dans Laravel

Laravel propose deux façons de capturer le paramètre passé :

- Paramètre requis
- Paramètre optionnel

Paramètres requis:

Les paramètres de route sont encapsulés dans des {} (accolades) avec des alphabets à l'intérieur. Prenons un exemple où vous devez capturer l'ID du client.

```
Route::get('emp/{id}', function ($id) {  
    echo 'Emp '.$id;  
});
```

Le mécanisme de routage dans Laravel

Paramètre optionnel

De nombreux paramètres ne restent pas présents dans l'URL, mais les développeurs ont dû les utiliser.

Ces paramètres sont donc indiqués par un " ?" (point d'interrogation) après le nom du paramètre.

Example:

```
Route :: get ('emp/{desig?}', function ($desig = null) {  
    echo $desig; });  
Route :: get ('emp/{name?}', function ($name = 'Guest') {  
    echo $name; });
```

Les contrôleurs

Les contrôleurs sont une autre fonctionnalité essentielle fournie par Laravel. Au lieu de définir la logique de traitement des demandes sous la forme de fermetures dans les fichiers de route, il est possible d'organiser ce processus à l'aide de classes de contrôleurs. Que font les contrôleurs ? Les contrôleurs sont destinés à regrouper la logique de traitement des demandes associées dans une seule classe. Dans votre projet Laravel, ils sont stockés dans le répertoire **app/Http/Controllers**. La forme complète de MVC est Model View Controller, qui dirige le trafic entre les vues et les modèles.

Création de contrôleurs

Ouvrez votre cmd ou terminal et tapez la commande :

php artisan make:controller Controller-Name> --plain

Remplacez ce <nom-du-contrôleur> dans la syntaxe ci-dessus par votre contrôleur. Cela va éventuellement faire un constructeur simple puisque vous passez l'argument --plain.

Création de contrôleurs

Le contrôleur que vous avez créé peut être invoqué à partir du fichier **routes.php** en utilisant la syntaxe ci-dessous :

`Route::get('base URI', 'controller@method');`

Un exemple de code de contrôleur de base ressemblera à ceci, et vous devez le créer dans un répertoire tel que **`app/Http/Controller/AdminController.php`** :

Exemple

```
<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class AdminController extends Controller
{
    //
}
```

contrôleurs middlewares

Vous pouvez assigner des contrôleurs aux middlewares à router dans les fichiers de route de votre projet en utilisant la commande ci-dessous :

```
Route::get('profile', 'AdminController@show')->middleware('auth');
```

contrôleurs middlewares

Les méthodes middleware du contrôleur permettent d'affecter facilement un middleware à l'action et à l'activité du contrôleur. Des restrictions sur l'implémentation de certaines méthodes peuvent également être fournies aux middlewares sur la classe du contrôleur.

```
class AdminController extends Controller
{
    public function __construct()
    {
        // function body
    }
}
```

contrôleurs middlewares

En utilisant les closures, les contrôleurs peuvent permettre aux développeurs de Laravel d'enregistrer des middleware.

```
$this->middleware(function ($request, $next) {  
    // middleware statements;  
    return $next($request);  
}  
);
```

contrôleurs de ressources

La route de ressources de Laravel permet aux routes classiques "CRUD" pour les contrôleurs d'avoir une seule ligne de code. Ceci peut être créé rapidement en utilisant la commande make : controller (commande Artisan) quelque chose comme ceci".

php artisan make:controller PasswordController --resource

Le code ci-dessus produira un contrôleur dans l'emplacement **app/Http/Controllers/** avec le nom de fichier **PasswordController.php** qui contiendra une méthode pour toutes les tâches disponibles des ressources.

contrôleurs de ressources

Les développeurs Laravel ont également la possibilité d'enregistrer plusieurs contrôleurs de ressources à la fois en passant un tableau à une méthode de ressource, comme ceci :

```
Route::resources([
    'password' => 'PasswordController',
    'picture' => 'DpController'
]);
```